

Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: EJERCICIO 19

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Implementación y Código Fuente

El código fuente demuestra la capacidad del autómata para no solo reconocer tokens matemáticos complejos, sino para aplicar conversiones de base (operaciones algorítmicas) utilizando la biblioteca nativa de C en tiempo de ejecución.

- `#include <stdlib.h>`: Directiva requerida para utilizar `strtol()` (String to Long).
- `0[xX][0-9a-fA-F]+`: Expresión regular estructurada para hexadecimales. Exige el prefijo literal `0x` o `0X` seguido obligatoriamente (+) de una mezcla de dígitos decimales o letras de la A a la F.
- `strtol(yytext, NULL, 16)`: Función que recibe la cadena literal detectada, asume su base matemática como 16 y la transforma directamente en un valor entero operado por el procesador.
- Reglas `[\t\n]+` y `./`: Actúan como filtros para ignorar texto genérico y centrarse solo en los hexadecimales.

Tipos de entradas que recibe: Literales numéricos válidos en base 16 (ej. `0xFF`, `0x1A3F`, `0X10`) incrustados dentro de texto natural o código.

Cadenas con las que el código fallaría (Errores Léxicos):

- Cadena `1A3F` (sin prefijo): A pesar de ser una combinación válida en base 16, al no tener el iniciador `0x`, la regla no logrará una coincidencia y descartará silenciosamente la cadena a través de la regla comodín `(.)`.
- Cadena `0x1A3G`: El analizador clasificará correctamente la subcadena `0x1A3` como hexadecimal y aplicará la matemática. Sin embargo, al llegar a la letra `G`, interrumpirá la lectura de ese token y dejará la `G` abandonada como un error suelto, alterando la semántica deseada.

Código fuente del archivo `diecinueve.l`:

}

```
fmerino@fmerino: ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECIN...
GNU nano 8.4 diecinueve.l *
%{
#include <stdio.h>
#include <stdlib.h> /* Necesaria para la funcion strtol() */
}%

%[xX][0-9a-fA-F]+ {
    /* Convertimos la cadena yytext (en base 16) a un entero largo decimal */
    long valor_decimal = strtol(yytext, NULL, 16);
    printf("HEXADECIMAL: %s => DECIMAL: %ld\n", yytext, valor_decimal);
}

[ \t\n]+ { /* Ignorar espacios y saltos de linea */ }
. { /* Ignorar cualquier otro texto */ }
}%

int main()
{
    yylex();
    return 0;
}
```

Archivo entrada.txt:

```
fmerino@fmerino: ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECIN..
GNU nano 8.4 entrada.txt
El valor original del enunciado es 0x1A3F
Tambien tenemos 0xFF y 0X10
Este numero 123 no debe ser detectado porque es decimal
```

2. Capturas de Ejecución

Se procesó un archivo de texto con múltiples sentencias que combinaban literales hexadecimales válidos, literales numéricos en base 10 y lenguaje natural.

Ejecución final: Prueba de Procesamiento y Redirección (Consola y Archivo salida.txt):

```
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ touch diecinueve.l entrada.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ nano diecinueve.l
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ nano entrada.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ nano entrada.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ lex diecinueve.l
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ gcc lex.yy.c -lfl
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ ./a.out < entrada.txt > salida.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ cat salida.txt
HEXADECIMAL: 0x1A3F => DECIMAL: 6719
HEXADECIMAL: 0xFF => DECIMAL: 255
HEXADECIMAL: 0X10 => DECIMAL: 16
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/DIECINUEVE$ █
```

Descripción de lo observado:

- **Conversión Matemática Asíncrona:** El autómata identificó exitosamente el literal complejo 0x1A3F y procesó su representación abstracta hasta convertirla en el número entero base-10 6719 antes de su impresión.
- **Discriminación Precisa de Patrones:** Números insertados intencionalmente en el archivo de prueba (como 123) que carecían del prefijo de formato 0x, fueron

rigurosamente descartados y no produjeron salidas erróneas ni falsos positivos de conversión.

3. Conclusión

La integración de las utilidades de C con la extracción de lexemas (yytext) demuestra que los escáneres léxicos son herramientas altamente dinámicas. Evaluar y traducir las bases de los literales constantes directamente durante la fase de tokenización permite entregar atributos semánticos resueltos a las fases subsecuentes del compilador, aliviando la carga computacional del analizador semántico.